

J A V A S C R I P T

Unit 2: Web Interaction

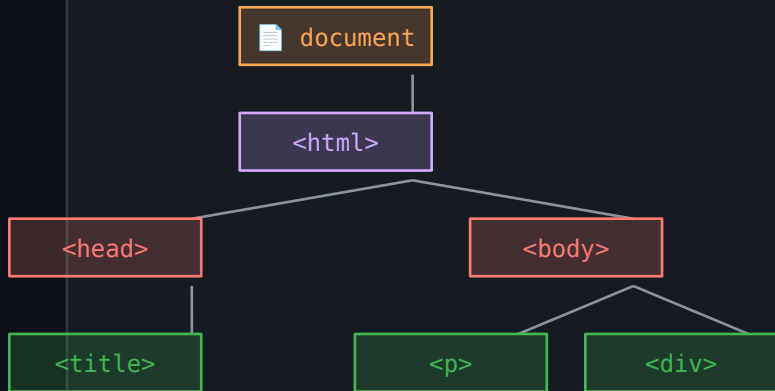
DOM · BOM · Events · Forms · Validation · Fetch API

Making web pages dynamic, interactive, and connected

The JavaScript logo, consisting of the letters 'JS' in a bold, black, sans-serif font, centered within a solid coral-colored square.

Document Object Model (DOM)

DOM Tree Structure



Selector Methods

`getElementById()`

Select by id attribute

`getElementsByClassName()`

Select by class name

`getElementsByTagName()`

Select by tag name

`querySelector()`

First match (CSS selector)

`querySelectorAll()`

All matches (CSS selector)

```
let el = document.getElementById("myDiv");
let items = document.querySelectorAll(".item");
el.style.color = "red";
```

DOM Manipulation

Content

- `innerHTML` — get/set HTML content
- `innerText` — get/set text only
- `textContent` — all text nodes
- `value` — form input values

Styling

- `element.style.property` — inline styles
- `classList.add()` — add CSS class
- `classList.remove()` — remove class
- `classList.toggle()` — toggle class

Attributes

- `getAttribute(name)`
- `setAttribute(name, value)`
- `removeAttribute(name)`
- `hasAttribute(name)`

Create & Insert

- `createElement(tag)`
- `appendChild(child)`
- `insertBefore(new, ref)`
- `removeChild(child)`

Styling Elements with JavaScript

Inline Style Properties

<code>element.style.color</code>	<code>"red" / "rgb(255,0,0)"</code>
<code>element.style.fontSize</code>	<code>"18px" / "1.2em"</code>
<code>element.style.fontWeight</code>	<code>"bold" / "700"</code>
<code>element.style.backgroundColor</code>	<code>"yellow"</code>
<code>element.style.display</code>	<code>"none" / "block" / "flex"</code>
<code>element.style.border</code>	<code>"1px solid black"</code>
<code>element.style.margin</code>	<code>"10px" / "10px 20px"</code>
<code>element.style.padding</code>	<code>"5px 10px"</code>

```
// Setting inline styles
let elem = document.getElementById("intro");
elem.style.color = "blue";
elem.style.fontSize = "18px";
elem.style.fontWeight = "bold";

// Reading computed style
let styles = window.getComputedStyle(elem);
console.log(styles.getPropertyValue("color"));
// → rgb(0, 0, 255)

// Using classList for CSS classes
elem.classList.add("highlight");
elem.classList.remove("hidden");
elem.classList.toggle("active");
if (elem.classList.contains("active")) {
  console.log("Element is active!");
}
```

Browser Object Model (BOM)



window

Global browser window. All BOM objects are properties of window.

- `window.innerWidth / innerHeight`
- `window.open() / window.close()`
- `alert(), confirm(), prompt()`



screen

Information about the user's screen resolution and color depth.

- `screen.width / screen.height`
- `screen.availWidth / availHeight`
- `screen.colorDepth`



location

Current URL and navigation. Use to redirect or read URL parts.

- `location.href` – full URL
- `location.pathname` – path
- `location.assign()` – navigate



history

Browser session history. Navigate forward and backward.

- `history.back() / forward()`
- `history.go(n)`
- `history.length`



navigator

Info about the user's browser and OS environment.

- `navigator.userAgent`
- `navigator.language`
- `navigator.onLine`



timers

Schedule code to run after a delay or at regular intervals.

- `setTimeout(fn, ms)`
- `setInterval(fn, ms)`
- `clearTimeout() / clearInterval()`

JavaScript Events

Mouse Events

- click
- dblclick
- mouseenter
- mouseleave
- mouseover
- mouseout

Keyboard Events

- keydown
- keyup
- keypress

Form Events

- submit
- change
- input
- focus
- blur
- reset

Window Events

- load
- resize
- scroll
- unload

```
// addEventListener – preferred approach
document.getElementById("btn").addEventListener("click", function(event) {
  console.log("Clicked!", event.target);
  event.preventDefault(); // Stop default behavior
});
```

Form Validation



Required Field

```
if (name === "") { alert("Required!"); return false; }
```



Email Format

```
/^[\\w.-]+@[\\w.-]+\\.\\w{2,}$/.test(email)
```



Age Range

```
age >= 18 && age <= 100 && !isNaN(age)
```



Password Strength

```
/(?=.*\\d)(?=.*[a-z])(?=.*[A-Z]).{8,}/.test(pwd)
```

```
function validateForm() {  
  const name = document.getElementById("name").value;  
  const email = document.getElementById("email").value;  
  const age = document.getElementById("age").value;
```

Fetch API & Async JavaScript

Promises

A Promise represents an async operation with 3 states: pending → fulfilled or rejected.

async/await

Syntactic sugar over Promises. Write async code that reads like synchronous code.

fetch()

Modern API to make HTTP requests. Returns a Promise resolving to a Response object.

JSON

JavaScript Object Notation — standard data format for APIs. Use `response.json()` to parse.

```
// fetch() with .then() chain (Promise-based)
fetch("https://api.exchangerate-api.com/v4/latest/INR")
  .then(response => response.json())
  .then(data => {
    const rate = data.rates["USD"];
    console.log("1 INR = " + rate + " USD");
  })
  .catch(error => console.error("Error:", error));

// async / await (cleaner syntax)
async function getRate(currency) {
```

Project: INR Currency Converter

01

HTML Structure

Input field, currency select dropdown, Convert button, result div

02

CSS Styling

Flexbox centering, card layout, button hover states, responsive design

03

Event Listener

click event on Convert button triggers conversion logic

04

Fetch API

GET request to exchangerate-api.com/v4/latest/INR endpoint

05

DOM Update

Display converted amount using `document.getElementById("result")`

06

Error Handling

`.catch()` handles network errors with user-friendly alert

Unit 2 — Quick Reference Cheat Sheet

DOM

- Select: `getElementById`, `querySelector`
- Modify: `innerHTML`, `style`, `classList`
- Create: `createElement`, `appendChild`

BOM

- window: `alert`, `confirm`, `prompt`
- location: `href`, `assign`, `pathname`
- navigator: `userAgent`, `onLine`

Events

- `addEventListener(type, handler)`
- `event.preventDefault()`
- Mouse, Keyboard, Form, Window

Validation

- Required field check
- Regex for email/phone
- Range checks for numbers

Fetch API

- `fetch(url).then().catch()`
- `async/await` with `try/catch`
- `response.json()` to parse

Timers

- `setTimeout(fn, ms)`
- `setInterval(fn, ms)`
- `clearTimeout` / `clearInterval`